UNIVERSITY OF NIŠ
FACULTY OF ELECTRONIC ENGINEERING



Vedran Mitić 2123

AI Model as a Search Tool for Medicine Package Leaflets

2026.

List of abbreviations used:

AI – Artificial Intelligence

NN – Neural Network

NLP – Natural Language Processing

LLM – Large Language Model

PEFT – Parameter-Efficient Fine-Tuning

Contents

Introduction 4
1. **The Question Answering Task** 5
2. **The BERT Model** 7
   2.1. **Sentence-BERT and Vector Representations** 9
   2.2. *MPNet* 11
3. **Large Language Models** 14
4. **Parameter-Efficient Fine-Tuning** 16
   4.1. LoRA (Low-Rank Adaptation) 16
   4.2. **Quantization** 17
   4.3. QLoRA 17
5. RAG 20
6. **Specifics of the Serbian Language and Text Preprocessing** 22
   6.1. **Transliteration** 22
   6.2. **Tokenization** 23
   6.3. **Lemmatization and Morphosyntactic Meaning** 23
7. **Practical Part** 25
   7.1. **Data Collection and Preprocessing** 26
   7.2. **Vector Database Creation** 27
   7.3. **QLoRA Fine-Tuning** 29
   7.4. **System Prompt** 30
   7.5. **Usage Examples** 31
   7.6. **Problems** 38
   7.7. **Numerical Evaluation** 41
   7.8. **Comparison with the Original Model** 42
8. **Conclusion** 45
References: 46

Introduction

The idea of communication between humans (users) and computers (devices) — or rather, the idea of facilitating such communication — has been one of the fundamental aspirations of computer scientists since the earliest pioneering days of the technology. Human–computer interaction thus evolved, from cable-switching to the direct use of natural language, as we see today in intelligent systems with integrated artificial intelligence (AI) models capable of "understanding" language. Although AI experienced its first major boom in the early 2010s in the field of computer vision, where various modifications of neural networks (NN) achieved near-perfect accuracy on numerous benchmark tests, it can be argued that the revolution in natural language processing (NLP) took place in 2017, with the emergence of the now well-known "Transformer" architecture, which forms the basis of large language models (LLMs). These models, distinguished by their ability to extract and understand context over large input sequences, as well as to generate text, may represent the pinnacle of the aforementioned human communication. The present — and indeed the future — is entirely unimaginable without the implementation of AI models in the various devices that surround us, with the ability to communicate through human speech, whether via textual or audio input/output.

Due to the enormous corpus of data on which they are trained and their scaling requirements, LLMs are hardware-expensive and cumbersome, so the current challenges in LLM design lie primarily in how to compress them efficiently (knowledge compression), as well as in achieving faster and better knowledge extraction and adaptation to local problems[1]. In addition, since the most widely known and most frequently used LLMs are trained predominantly on English-language corpora[2], it is necessary to find solutions for adapting them to lower-resource languages such as Serbian (among others). This paper addresses a question-answering model for the Serbian language, using RAG and LoRA technologies for knowledge extraction and linguistic adaptation.

[1] Here I refer to what is called a "downstream" task in the literature. This includes all groups of NLP problems — e.g., text translation, summarization, classification, etc. It can also be viewed as adaptation to local datasets that were not part of the original training set during pre-training. In practical terms, this is fine-tuning.

[2] These models, of course, also have the ability to use languages other than English, but the question is to what extent the quality of answers in a lower-resource language matches that in English. My assumption is that this quality is at a lower level.

1. The Question Answering Task

The task of question answering (QA) represents one of the most important and most complex domains within NLP. The fundamental goal of a QA system is to provide the user with an accurate, direct, and contextually appropriate answer to a query formulated in free-form natural language. Unlike traditional search engines, which return a list of entire documents or web pages based on keyword matching, modern QA systems perform semantic analysis of the query, search relevant knowledge sources, and generate a specific answer. What essentially distinguishes the QA task from other standard NLP tasks (such as text classification, translation, or summarization) is a specific set of requirements and constraints, such as the primacy of factuality over creativity. This does not mean that creativity is entirely excluded from LLMs — QA tasks can be extractive, generative, closed-domain, and open-domain.

Early QA systems relied on hand-written rules, regular expressions, ontologies, and lexical databases (such as WordNet). These systems were extremely rigid, difficult to scale, and unsuccessful at processing unstructured language outside a narrowly defined domain. With further algorithmic improvements, QA systems began using word frequency to locate relevant documents. However, these methods suffer from the problem of synonymy (they fail to recognize that the words “lek” and “medikament” [medicine/drug] carry the same meaning) and from syntactic variation. The revolution began with the emergence of encoder models such as BERT and RoBERTa. Dual encoders such as DPR (Dense Passage Retriever) and SBERT (Sentence-BERT) were introduced, translating questions and text passages into dense semantic vectors within a shared vector space. Such retrievers enable efficient search of large databases and can supply passages to either extractive or generative QA systems. With increased model capacity and the adoption of decoder architectures (such as the Llama family), generative QA has become increasingly prominent. To overcome the limitations of parametric memory alone, the concept of Retrieval-Augmented Generation (RAG) was developed. RAG combines the parametric memory of a generative LLM with non-parametric memory (a dynamic vector database of documents), allowing the system to retrieve accurate facts in real time and generate precise answers based on them.

Despite the remarkable progress achieved by modern architectures, the field of question answering still faces several fundamental challenges:

• Hallucinations: Large language models are, at their core, statistical next-token generators. When asked a question for which they lack sufficient parametric knowledge, they tend to generate answers that appear correct but are factually entirely inaccurate. In expert applications (e.g., medicine and pharmacy), hallucinations represent an unacceptable risk.

• Challenges for lower-resource languages: The most powerful LLMs in the world are trained predominantly on English-language corpora. Applying these models to languages with fewer available resources, such as Serbian, brings specific problems:

– Complex morphology: Serbian has rich case declension and verb conjugation, which complicates standard tokenization.

– Biscriptality (Cyrillic–Latin): models must handle both scripts equally effectively and map them into a unified semantic space.

– Tokenizer inefficiency: LLMs often split words of lower-resource languages into a large number of small subwords, which increases sequence length, slows generation, and consumes more computational memory.

• Knowledge staleness and control: the knowledge embedded in a model's parameters is static and limited to the moment pre-training was completed, while global knowledge changes on a daily basis.

• Context-window limitations and chunking: although modern LLMs support increasingly larger input context windows, feeding excessively large amounts of unstructured text into the prompt drastically increases latency and cost and raises the risk of the so-called “lost in the middle” phenomenon. For this reason, the development of advanced text-chunking strategies (e.g., header-based chunking), which preserve metadata and the structural integrity of documents, is of critical importance for a successful QA system.

• Evaluation challenges: while extractive QA is easily evaluated using mathematical metrics such as Exact Match or F1 score, evaluating freely generated text requires considerably more complex — and perhaps more subjective — methods.

The practical part of this paper uses both an encoder and a decoder model to implement the question-answering system. While the decoder model is used as the LLM for text (answer) generation, the encoder model is used to create vector representations of text for the vector knowledge base employed by the RAG mechanism, as well as for the retriever.

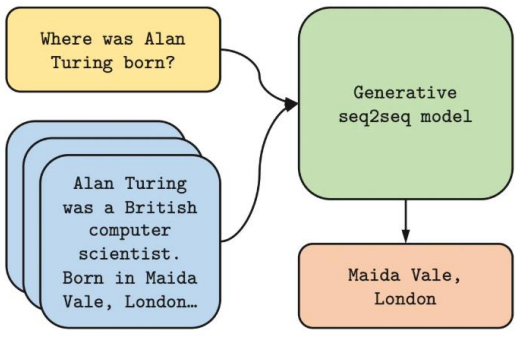


Figure 1. Basic example of a QA system [11]

2. The BERT Model

As discussed in the previous chapter, the development of modern question-answering systems is grounded in the need for a system to precisely localize information within a text or a relevant document. Traditional architectural approaches lacked the ability to simultaneously observe both the left and right context embedded in deep word representations. The emergence of the BERT (Bidirectional Encoder Representations from Transformers) model marked a turning point in the extractive QA task, enabling dense semantic modeling of the relationship between the query (question) and the context (the text containing the answer). BERT relies exclusively on the encoder portion of the original Transformer architecture (Figure 2). The model's core innovation is bidirectionality: unlike classical unidirectional language models that predict context left-to-right (or right-to-left), BERT analyzes the entire sequence of tokens simultaneously through a multi-layer self-attention mechanism.

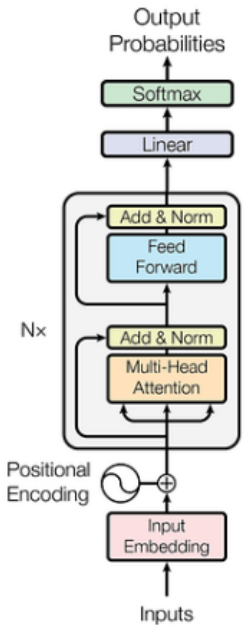


Figure 2. Transformer encoder architecture

BERT's input representation is formed as the sum of three separate vectors for each token in the sequence (Figure 3): (1) Token Embedding — a vector representation of the token itself from the vocabulary (WordPiece tokenization is used); (2) Segment Embedding — distinguishes whether a token belongs to the first sequence (e.g., the question, marked E_A) or the second sequence (e.g., the context passage, marked E_B); (3) Positional Embedding — adds information about the token's absolute position in the sequence, since the attention mechanism itself has no built-in notion of order.

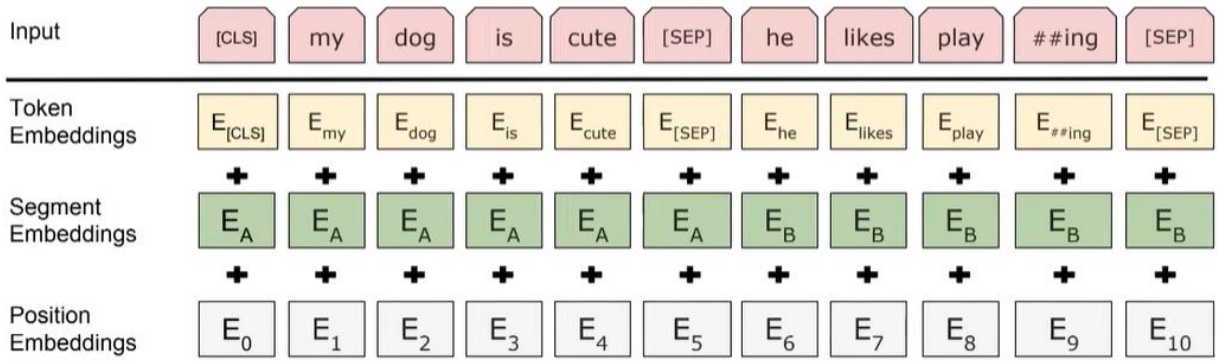


Figure 3. Vector representation of tokens in the BERT model

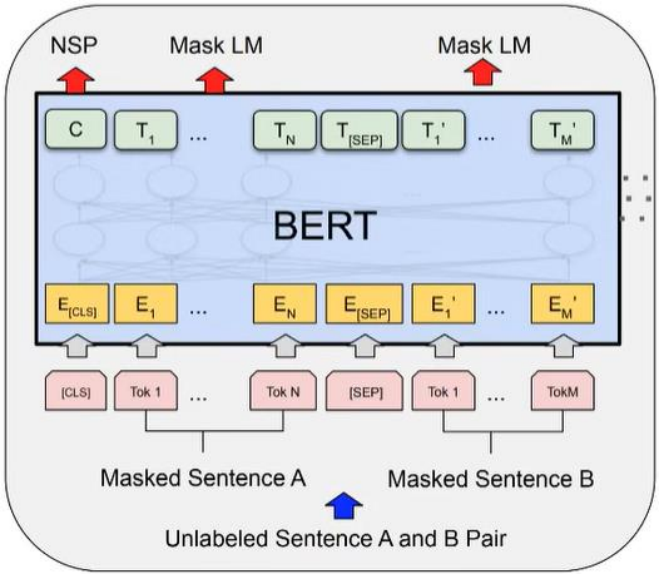


Figure 4. BERT pre-training

For pre-training, BERT uses two self-supervised tasks [1]: (1) Masked Language Modeling (MLM) — 15% of token positions are selected as prediction targets; of these, 80% are replaced with [MASK], 10% with a random token, and 10% remain unchanged. The model predicts the original tokens from their bidirectional context. (2) Next Sentence Prediction (NSP) — given sentence pairs (A, B), the model predicts whether B actually follows A in the source text or was sampled independently. This objective learns relationships between sentences.

The input to the QA system takes the form [CLS] Question [SEP] Context [SEP]. The [CLS] token represents a global representation of the entire pair, while [SEP] is used as a sequence separator (Figure 4).

The core of BERT's encoder functioning is scaled dot-product attention. For given query (Q), key (K), and value (V) matrices, the computation is defined as:

$$Attention(Q,K,V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad (1)$$

where d_k represents the dimensionality of the keys, used for scaling to prevent gradient saturation at high dimensions. To allow the model to simultaneously process information from different representation subspaces, Multi-Head Attention is used:

$$MultiHead(Q,K,V) = Concat(head_{1,\dots,}head_h)W^O \quad (2)$$

where $head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$, and W_i^Q, W_i^K, W_i^V and W^O are learned projection matrices. Optimization during MLM pre-training is based on cross-entropy over the masked tokens *M*:

$$\mathcal{L}_{MLM}(\theta) = -\sum_{i \in M} \log P(y_i | X_{masked}; \theta) \quad (3)$$

where y_i represents the actual token at position *i*, and X_{masked} is the masked token sequence.

In RAG-based question-answering systems (Chapter 5), BERT-based encoders can serve as retrievers, as in the original DPR-based RAG architecture [2]. M[3]ore recent systems often use specialized sentence-embedding models trained for semantic retrieval.

2.1. Sentence-BERT and Vector Representations

Despite all its improvements, using a standard BERT model (or RoBERTa) to compute semantic similarity across thousands of documents in real time is computationally extremely expensive. Forming pairs through a Cross-Encoder requires O(M · N) passes through the network. On the other hand, the [CLS] vector of a plain BERT model, without additional fine-tuning, often performs poorly when cosine similarity between sentences is measured directly. To overcome this problem and enable scalable vectorization of text into high-dimensional spaces, Sentence-BERT (SBERT) was developed [3]. SBERT adds a pooling layer and fine-tunes the network through a Siamese/triplet architecture, so that the resulting sentence-vector representation becomes semantically meaningful and suitable for direct comparison using cosine similarity.

A Siamese network is an architecture consisting of two identical neural networks that share exactly the same weights and parameters; instead of concatenating two sentences into a single input, one sentence (A) is passed through the first network while the other sentence (B) is

[3] Although it achieved significant progress, the classic BERT model has structural limitations in the context of complex QA and RAG systems: sequence length, the inefficiency of the NSP task, etc. This paper devotes a chapter to this model for two reasons: (1) to describe the basic idea of the Transformer architecture (also used in decoder models) and of encoder models (embedding creation); (2) because the original RAG paper bases its retriever precisely on this model.

passed through the second. In this way, any parameter update during training applies equally to both sides, and the resulting output vectors are compared using cosine similarity.

A triplet network is an extension of the Siamese architecture that uses three identical networks with shared weights and simultaneously observes three inputs: a reference sentence (Anchor, a), a positive sentence (p) that is semantically similar or related to the anchor, and a negative sentence (n) that is different from or unrelated to the anchor. By computing the loss function at the output and optimizing it, the network is forced to make the distance between a and p smaller than the distance between a and n by at least a defined margin epsilon:

$$\max(\|s_a - s_p\| - \|s_a - s_n\| + \epsilon, 0) \quad (4)$$

where s_x represents the vector representation of sentence x. By the end of training, the vector space is organized so that semantically close sentences lie near one another, while dissimilar ones are kept at a certain distance.

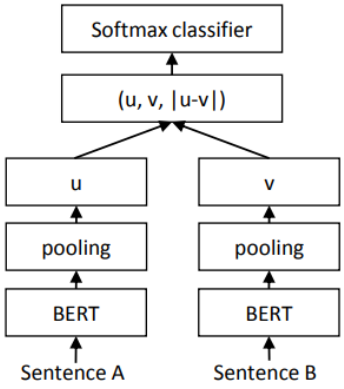


Figure 5. SBERT Siamese network architecture [3]

The process within SBERT unfolds in three steps (Figure 5): (1) the sentence is split into tokens and passed through the encoder model's layers, so each output token receives its own individual vector; (2) pooling — SBERT takes all output token vectors and combines them into a single fixed vector for the whole sentence using mean pooling[4]; (3) fine-tuning — the encoder's parameters are adjusted during training through the Siamese or triplet structure, so that the resulting sentence vector accurately reflects the meaning of the entire sentence. After fine-tuning, the final layers (classification or regression heads) are removed, and at inference time only the adapted embedding model with the pooling layer is used to obtain sentence embeddings (Figure 6). A model trained in this way is capable of extracting the essence of a sentence and placing it in the vector space close to semantically similar sentences.

[4] There are also strategies using MAX pooling (maximum across dimensions) and the previously mentioned [CLS] token (taking the vector of the first token), but work [3] empirically demonstrated that MEAN pooling achieves the best results.

In an information-retrieval system with a large document base, a BERT cross-encoder must process each query–document pair separately, making exhaustive scoring expensive. With SBERT, document embeddings can be computed once during offline indexing and reused. At query time, only the query embedding needs to be computed; actual search latency depends on the corpus, index, and hardware. That vector is then compared against the pre-stored vectors from the database using cosine similarity or specialized vector-search indices, and the result consists of the documents whose vector is closest to the query vector.

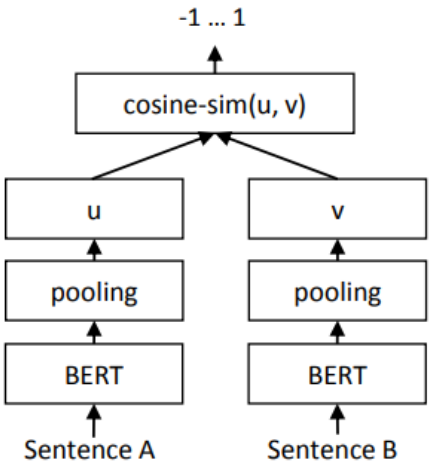


Figure 6. SBERT architecture at inference [3]

2.2. *MPNet*

Within the Sentence-BERT framework discussed in Section 2.1, embedding quality depends on the encoder architecture and its training objective. The practical implementation (Chapter 7) uses sentence-transformers/paraphrase-multilingual-mpnet-base-v2 [12]. Despite its name, this multilingual checkpoint uses an XLM-RoBERTa backbone, as specified in its published configuration. It should therefore be distinguished from the original MPNet architecture. The following discussion presents MPNet (Masked and Permuted Pre-training, Figure 7) [4] as theoretical background.

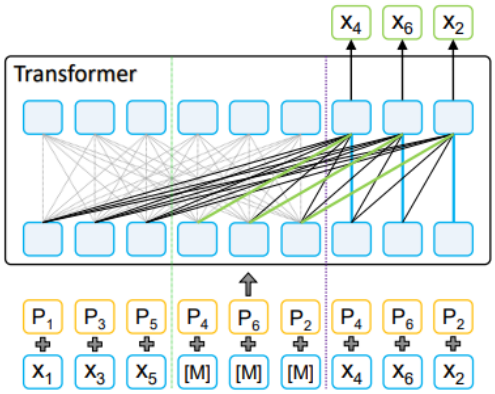


Figure 7. MPNet structure [4]

To understand the innovativeness of the MPNet architecture, it is necessary to analyze two fundamental pre-training approaches for encoder models that preceded it: MLM (Masked Language Modeling), applied in BERT, and PLM (Permuted Language Modeling), applied in XLNet. As already noted in Chapter 2, the MLM approach masks a percentage of tokens and attempts to predict them based on the remaining tokens, optimizing the following loss function [4]:

$$\mathcal{L}_{MLM}(\theta) = \mathbb{E}_{z \in Z_n}\left[\sum_{t=c+1}^{n} \log P(x_{z_t} | x_{z_{\leq c}}, M_{z_{>c}}; \theta)\right] \tag{5}$$

where $M_{z_{>c}}$ represents the set of masked tokens at position $z_{>c}$. Formula (5) shows that MLM assumes that all masked tokens are mutually independent given the unmasked context, which is very often not the case in practice, leading to the neglect of mutual semantic dependency among masked tokens.

On the other hand, to overcome this problem, XLNet introduces Permuted Language Modeling (PLM). For a given token sequence of length n, all possible permutations of the index order Z_n are considered, and autoregressive prediction is performed in the sampled permutation order [4]:

$$\mathcal{L}_{PLM}(\theta) = \mathbb{E}_{z \in Z_n}\left[\sum_{t=c+1}^{n} \log P(x_{z_t} | x_{z_{<t}}; \theta)\right] \tag{6}$$

where z_t denotes the *t*-th element in permutation z, $z_{<t}$ all elements preceding position *t* in that permutation, and *c* the number of unmasked tokens. However, this approach loses information about the position of tokens that come after it in the permutation — i.e., about the context of the whole sentence.

The MPNet architecture (Figure 7) synthesizes the useful properties of both approaches within a unified mathematical framework and defines its objective function as:

$$\mathcal{L}_{MPNet}(\theta) = \mathbb{E}_{z \in Z_n}\left[\sum_{t=c+1}^{n} \log P(x_{z_t} | x_{z_{<t}}, M_{z_{>c}}; \theta)\right] \tag{7}$$

In other words, when predicting token x_{z_t}, *MPNet* uses all previously predicted tokens in the permuted sequence $x_{z_{<t}}$ (preserving mutual dependency of output tokens) and all positional information about the masked tokens from the whole sentence $M_{z_{>c}}$ (preserving positional knowledge).

To enable efficient training according to (7), MPNet implements an extended two-stream self-attention mechanism together with the introduction of so-called position compensation [4]. The architecture maintains two parallel information streams within the transformer layers:

- Content Stream: receives both the tokens and their absolute positions $x_i + P_i$. Its role is to extract a full contextual representation of the unmasked part of the text.
- Query Stream: receives only the positional embeddings and the masked tokens ($[M] + P_i$), not the actual content of the tokens being predicted. Its task is to predict the token at a given

position based exclusively on previously available tokens and the overall positional structure.

Position compensation within the attention matrix ensures that, even within the Query Stream, during every step of predicting token x_{z_t}, the model is passed the positional embeddings of all n tokens of the original sentence. In this way, MPNet uses 100% of the positional information of the whole sequence while simultaneously preserving the autoregressive modeling of the output.

3. Large Language Models

Unlike encoder models (such as BERT and MPNet), which are primarily designed for bidirectional semantic analysis and the formation of vector representations, modern LLMs rely predominantly on the decoder-only transformer architecture (Figure 8). The fundamental task of decoder models is text generation through autoregressive prediction of the next token, based on all previously generated tokens and the input prompt.

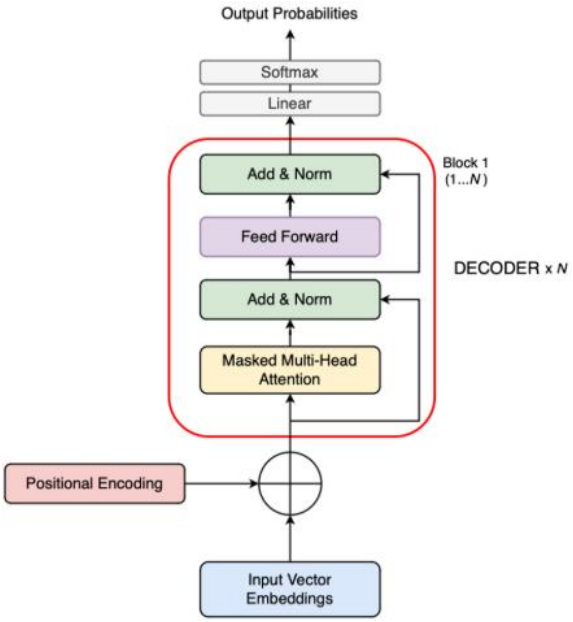


Figure 8. Architecture of a typical decoder model [13]

The architecture of modern decoders omits the cross-attention layer characteristic of the original encoder–decoder architecture and uses exclusively causal self-attention. This mechanism prevents the model, while analyzing and predicting the current token, from seeing future tokens in the sequence, thereby strictly preserving the causal (autoregressive) nature of generation. Scaled dot-product attention with a causal mask is defined by adding a causal mask matrix M to formula (1):

$$Attention(Q,K,V) = softmax(\frac{QK^T}{\sqrt{d_k}} + M)V \tag{8}$$

where M has the elements:

$$M_{ij} = \begin{cases} 0, i \geq j \\ -\infty, i < j \end{cases}$$

Applying the mask M ensures that, for any position i, the attention weights toward positions $i > j$ become zero after applying the *Softmax* function. The probability of generating the entire output token sequence $Y = (y_1, y_2, \ldots, y_N)$ for a given input context is computed as the product of the conditional probabilities of all individual tokens, while optimization is based on straightforward cross-entropy minimization at the output.

The challenges of this type of model are very similar to those already discussed for encoder models. However, due to their much greater prevalence and scalability, today's LLMs — even those publicly available as open weights — require enormous hardware capacity to run, making accessibility one of the major problems for the broader community[5]. In the practical part of this paper (Chapter 7), a decoder LLM takes on the central role of answer generator within the RAG system. Unlike earlier extractive QA systems, in which the answer is simply extracted from the retrieved passage, this paper uses a modern decoder model (Llama-3.1-8B-Instruct [15]), whose basic task is context synthesis.

[5] The development and operation of LLMs by individual users outside major centers is very limited due to the required resources.

4. Parameter-Efficient Fine-Tuning

To overcome the described hardware constraints and enable the practical application of large language models (LLMs) on local infrastructure, Parameter-Efficient Fine-Tuning (PEFT) methods have been developed. Instead of retraining and updating billions of parameters of a pre-trained model, PEFT methods enable adapting a system to specific, local tasks by optimizing only a small percentage of newly introduced or selected parameters, drastically reducing VRAM usage and storage costs.

4.1. LoRA (Low-Rank Adaptation)

Traditional full fine-tuning starts from the initialized weights of a pre-trained model Φ_0 and, during training, updates all of its parameters by an amount $\Delta\Phi$. The main drawback of this approach is that a completely new set of all parameters — whose dimensionality equals that of the original model —($|\Delta\Phi| = |\Phi_0|$) must be trained and stored for every new individual task [5]. For modern LLMs, such as the Llama-3.1-8B model used in Chapter 7, this creates two major problems: (1) heavy VRAM demand for storing gradients and optimizer states, including first and second moments; (2) enormous storage requirements for the models themselves and their deployment. Although the pioneering approach of freezing most layers and fine-tuning only the last few layers is still used in practice, this can limit adaptation quality in transformer architectures, since knowledge and contextual logic are distributed across all layers of the network.

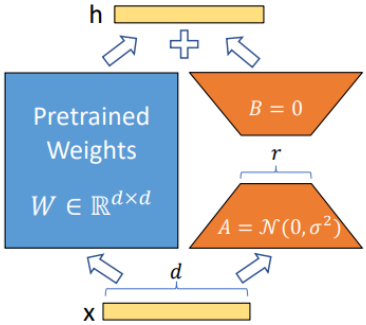


Figure 9. LoRA structure [5]

LoRA (Low-Rank Adaptation) overcomes this problem by relying on the low intrinsic dimension hypothesis. According to this hypothesis, although the weight matrices in pre-trained models are dense and full-rank within a high-dimensional space, the change in weights ΔW during adaptation to a specific task essentially occurs within a much smaller subspace — i.e., it has a low intrinsic rank ($r \ll \min(d, k)$). Instead of directly updating the original weight matrix $W_0 \in \mathbb{R}^{d\times k}$, *LoRA* freezes the base weights W_0 and introduces a parallel branch consisting of two much smaller rank-decomposition matrices – A and B (Figure 9).

For an input vector x, the modified forward pass through a LoRA layer is simply computed as the coordinate-wise sum of the output of the frozen branch and the output of the parallel branch:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} BAx \tag{9}$$

where $W_0 \in \mathbb{R}^{d\times k}$ is the frozen matrix of original weights, $A \in \mathbb{R}^{r\times k}$ is a decomposition matrix initialized with a Gaussian distribution, $B \in \mathbb{R}^{d\times r}$ is a decomposition matrix initialized with zeros (so that the first training step equals the original model output $W_0 x$ thereby preserving the initial model behavior), r is the low rank of the adapter matrices such that $r \ll \min(d, k)$ and α is a scaling hyperparameter. Scaling by $\frac{\alpha}{r}$ is introduced because changing the rank also changes the number of parameters being updated, so this scaling controls the magnitude of the actual change ΔW and can reduce the need to retune the learning rate across ranks [5].

In this way, LoRA has become one of the most innovative methods in deep learning, drastically reducing the number of trainable parameters by over 99% without introducing additional inference latency. After training is complete, the product ΔW can be explicitly added mathematically to the frozen weights W_0. Consequently, during production use, the system performs no additional matrix operations and experiences no slowdown relative to the base model. Furthermore, to store a model adapted to a specific local task, it is sufficient to store only the trained low-rank factors A and B that define ΔW instead of the whole cumbersome model [5].

4.2. Quantization

Quantization reduces a model's memory footprint by representing weights at lower precision, for example using 8-bit integers or 4-bit formats such as INT4 and NF4. It can also accelerate computation when supported by the hardware and implementation. Quantization is widely used for inference, and quantization-aware training also predates QLoRA. QLoRA enables memory-efficient adaptation by keeping the quantized base weights frozen while training higher-precision low-rank adapters [6].

4.3. QLoRA

QLoRA (Quantized Low-Rank Adaptation) is an advanced PEFT method that enables efficient fine-tuning of large language models in 4-bit precision while achieving performance comparable to full 16-bit fine-tuning in the experiments reported in [6]. The basic principle of the QLoRA system is based on a strict separation between the storage data type and the computation data type (Figure 10).

Throughout the entire training process, the weights of the base pre-trained model W remain completely frozen and are stored in VRAM in 4-bit format for maximum resource savings. When the input X reaches a given linear layer, the 4-bit weights of that layer are dynamically dequantized into a high-precision 16-bit format (usually BFloat16 or Float16) for computation. In this way, the quantized linear operations use a higher-precision computation format to maintain numerical stability. During the backward pass, gradients are neither computed nor stored for the 4-bit weights of the base model; instead, the error signal is propagated through the dequantized weights and updates only the parameters of the 16-bit

LoRA adapters. By adding LoRA adapters to all[6] linear layers of the transformer architecture, the model gains additional capacity to adapt despite quantization.

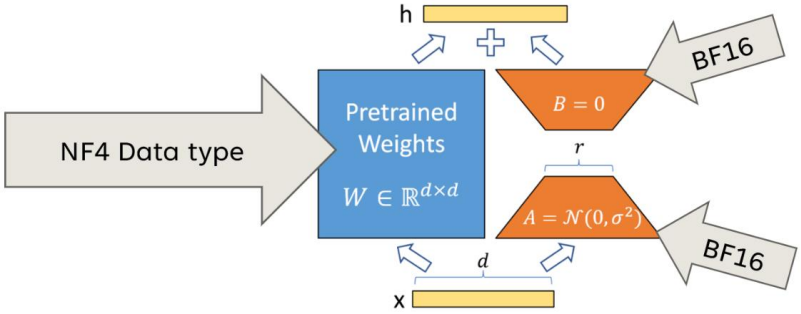


Figure 10. QLoRA compared to the LoRA structure [16]

Uniform INT4 quantization uses equally spaced levels, whereas FP4 uses a floating-point representation with non-uniform spacing. However, since most weights in a model cluster around zero, with only a very small number located at the extremes of the distribution, uniform spacing leads to substantial quantization error — equal space is spent on rare values as on the most common ones. For this reason, work [6] uses a specialized 4-bit data type called NormalFloat (NF4), which is adapted to the Gaussian distribution[7]. This data type divides the 4-bit space (16 possible states) so that each of the 16 possible ranges contains an equal expected number of weights. Because of this, the levels in NF4 are not equally spaced from one another — the quantization levels are denser around zero (where weight density is highest) and wider toward the extremes. In practical terms, NF4 can be viewed as a fixed lookup table of exactly 16 precomputed fixed numbers ("Appendix E" in [6]). It has been empirically demonstrated that this data type drastically reduces compression error compared with INT4 or FP4.

To prevent the influence of extreme values (outliers), which can disrupt the scaling of entire matrices, block-wise quantization is used when quantizing the model, whereby the weight matrix is divided into independent blocks of size B. For an input matrix X, the quantization process into a discrete format is defined as:

$$X^{quant} = round(c \cdot X) \tag{10}$$

where c is a scaling factor defined as $c = \frac{\max_val}{absmax(X)}$, in which $\max_val$ is the largest value of the range being quantized into (e.g., 127 for 8-bit integers). The reverse process, dequantization, is defined as follows:

$$X^{dequant} = \frac{X^{quant}}{c} \tag{11}$$

Work [6] also proposes double quantization to further reduce the memory cost of storing the 32-bit scaling factor — i.e., the number of bits per parameter. This method is nothing more than an

[6] The original LoRA work [5] reports experiments adapting query and value projection matrices. QLoRA [6] extends adaptation to all transformer linear layers and reports improved recovery of full-precision performance in its experiments.

[7] NF4 quantization levels are designed for approximately normally distributed weights; NF4 is a discrete representation, not itself a continuous Gaussian distribution.

additional application of quantization, but applied to the stored scaling factors. Finally, for a single linear layer in a quantized base model with an attached LoRA adapter, the forward pass through a QLoRA layer is computed by the formula:

$$Y^{BF} = X^{BF} \cdot doubleDequant(c_1^{FP32}, c_2^{FP8}, W^{NF4}) + X^{BF16} L_1^{BF16} L_2^{BF16} \quad (12)$$

where $doubleDequant(\cdot)$ is:

$$\text{dequant}\left(\text{dequant}\left(c_1^{FP32}, c_2^{k-bit}\right), W^{4bit}\right) = W^{BF16} \quad (13)$$

In the practical implementation (Chapter 7), QLoRA was used to fine-tune Llama-3.1-8B-Instruct on Serbian-language data. The base model used 4-bit NF4 quantization with double quantization and Float16 computation. This reduced the memory required for the base weights and enabled training on a single NVIDIA T4 GPU. The supplied notebook does not record peak training VRAM, so an exact total training-memory figure cannot be verified from the saved results. Generation quality is assessed separately in Sections 7.5–7.8.

5. RAG

Although large language models possess a remarkable ability to generate text and synthesize knowledge, their training on static datasets brings significant limitations: the inability to easily update knowledge in real time, difficulty tracing the provenance of information, and a tendency toward hallucination. To overcome these problems, the RAG (Retrieval-Augmented Generation) architecture was developed, combining parametric and non-parametric memory [2]. Parametric memory is essentially a classical pre-trained generative language model (originally BART; in modern systems this is a decoder LLM such as the Llama model used here), in which knowledge is implicitly encoded in the model's parameters. Non-parametric memory, on the other hand, is an external text index (most often a vector database consisting of tens of millions of document chunks); as such, it is separate from the model, accessible to users, and can be updated in real time as needed, without retraining the model itself.

The functioning of the model relies on the synergy of two components:

- *Retriever* (p_η) - its task is to find, for a given input query x, the K most relevant document chunks z from the external non-parametric memory using a vector-search index. In the original RAG paper, the retriever used is DPR (Dense Passage Retrieval), a bi-encoder structure consisting of two separate BERT models — one for the input query and one for documents [2]. Specialized sentence-embedding models (Section 2.1), including the multilingual checkpoint discussed in Section 2.2, are also used for retrieval. The relevance of document z to input x is scored using the dot product of their vector representations before normalization in the original RAG formulation [2]:
$$p_\eta(z|x) \propto \exp(d(z)^T q(x)) \quad (14)$$
Finding the *K* documents with the largest dot product reduces to finding the maximum inner product (*MIPS – Maximum Inner Product Search*).

- *The Generator* (p_θ) – an autoregressive model that receives a concatenated input composed of the input query x and the retrieved context z[8]. For each subsequent token y_i, the generator computes the probability $p_\theta = (y_i|x, z, y_{1:i-1})$ based on the previously generated tokens $y_{1:i-1}$, the input x and the retrieved chunk z.

In the original RAG training formulation [2], the training set contains only pairs (question x, answer y), with no label indicating which document z was actually used, so document z is treated as a hidden (latent) variable. To compute the overall answer probability $p(y|x)$, RAG performs marginalization (summing probabilities) over the *K* retrieved documents in two different ways [2]: (1) *RAG-Sequence* (marginalization over the entire sequence) assumes that a single document z is responsible for generating all tokens within the whole output sequence y. For each of the K retrieved documents, the model independently computes the probability of the entire answer (by

[8] Although the system prompt is not originally part of RAG technology per se, it is added to Figure 11 within the concatenated input, since it was used in Chapter 7.

multiplying the probabilities of all tokens), then sums these values weighted by the relevance of each document $p_\eta(z|x)$:

$$p_{RAG-Sequence}(y|x) \approx \sum_{z \in top-K(p(\cdot|x))} p_\eta(z|x) \prod_{i=1}^{N} p_\theta(y_i|x, z, y_{1:i-1}) \quad (15)$$

During decoding at inference time, RAG-Sequence runs standard beam search separately for each document z forming a candidate set Y; (2) *RAG-Token* allows different documents z to be responsible for different tokens within the same sentence. Here, at each step i of generating an individual token y_i the influences of all documents are first summed, and the resulting probabilities are then multiplied along the entire output sequence:

$$p_{RAG-Token}(y|x) \approx \prod_{i=1}^{N} \sum_{z \in top-K(p(\cdot|x))} p_\eta(z|x)\, p_\theta(y_i|x, z, y_{1:i-1}) \quad (16)$$

This allows the model, when answering complex questions, to combine facts from several different sources into a single answer (e.g., taking the first fact from one document and the second from another).

Modern production systems often combine dense (vector) and sparse (BM25) search, after which they use Cross-Encoder Reranker models to further reorder the retrieved documents before sending them to the LLM. In the practical part of this paper (Chapter 7), the system shown in Figure 11 was developed, as an assistant for searching Serbian-language medicine package leaflets. The implementation concatenates retrieved passages into the generation prompt; it does not implement the RAG-Sequence or RAG-Token training objectives described above.

6. Specifics of the Serbian Language and Text Preprocessing

NLP for the Serbian language carries specific challenges arising from its morphological complexity, biscriptality, and dialectal variation. While most early NLP tools and techniques were developed for English — which has a relatively fixed word order and impoverished inflection — applying the same principles to Serbian requires understanding its specific linguistic features. Compared with English and other Germanic or Romance languages, Serbian is distinguished by several key characteristics:

• Digraphia (biscriptality): Serbian commonly uses both Cyrillic and Latin scripts. Digital corpora contain both scripts, and individual documents or user inputs may mix them. NLP systems therefore need to handle both forms.

• Rich inflection and complex morphology: Serbian has seven cases in singular and plural, three grammatical genders, and complex verb conjugation. A single word (e.g., a noun or adjective) can have dozens of different forms depending on case, gender, and number, whereas in English a word has only a few forms.

• Flexible word order: Serbian allows substantial variation in word order to express emphasis and information structure. This flexibility creates additional variation in questions and in the passages that may answer them.

• Ekavian and Ijekavian pronunciation: within the same language area, the Ekavian and Ijekavian variants are used in parallel (e.g., *čovek/čovjek, mleko/mlijeko, primer/primjer*), which expands vocabulary and requires additional model adaptation.

• Non-standard web text and diacritics: in online communication and on social media, users often omit diacritical marks (*c, s, z, dj* instead of *č, š, ž, đ*), which leads to high ambiguity (e.g., "sisati" can represent either sisati or *šišati* when diacr*itics are omitted*).

6.1. Transliteration

Transliteration is the process of converting text from Cyrillic script to Latin script or vice versa. The main challenge in transliterating from Latin to Cyrillic is digraphs — Latin letter pairs representing a single Cyrillic letter (*lj, nj, dž for љ, њ, џ*). While transliteration from Cyrillic to Latin works through a deterministic mapping, including mappings from single Cyrillic letters to Latin digraphs, the opposite direction requires contextual understanding, so as not to mistakenly transliterate words in which the letters n and j, or d and ž, appear adjacent at a prefix or stem boundary (e.g., producing "ињекција" instead of "инјекција" from "injekcija"). In traditional preprocessing pipelines, transliteration into a single script (usually Latin) was applied as a

mandatory data-unification step prior to vectorization, in order to avoid vocabulary duplication [10].

6.2. Tokenization

Tokenization is the process of splitting raw text into basic units (tokens) — words, word parts, punctuation, and symbols — which are also the inputs of an LLM. Some of the specific challenges of Serbian tokenization include:

• Future tense I: Serbian includes both joined forms (e.g., "praviću" — I will make) and multiword forms (e.g., "ja ću praviti"). Non-standard and dialectal variants also occur in web text, increasing tokenization variability.

• Non-standard concatenated or split tokens (1:n and n:1 problems): on the web and in colloquial language, words are often merged (e.g., "neznam" → "ne znam," I don't know) or split into several parts for stylistic reasons [9].

• Tokenization rules: traditional Serbian tokenizers used complex rule-based solutions (such as the ReLDI tokenizer) to correctly handle punctuation, abbreviations, and web domains.

• Subword tokenization in modern models: modern transformer models use algorithms such as Byte-Pair Encoding (BPE, used in Llama 3) or WordPiece (used in BERT and BERTić), which split words into parts [7]. This is of considerable help given Serbian's rich inflectional system, because it represents unfamiliar word forms through smaller units, although those units do not necessarily coincide with stems or grammatical endings.

6.3. Lemmatization and Morphosyntactic Meaning

Lemmatization is the process of reducing different forms of the same word to its base form — the infinitive for verbs, the singular nominative for nouns, etc. Due to Serbian's rich morphology, lemmatization is far more complex than in English. For example, in morphosyntactic (POS/MSD) tagging, Serbian requires labeling not only part of speech but also gender, number, person, case, and verb subtype, among others. One of the more popular tools for lemmatizing South Slavic languages is CLASSLA-Stanza [8], which extends the Stanza neural NLP pipeline and incorporates additional resources and adaptations for these languages.

Although traditional NLP pipelines may include manual text preprocessing (removing stop words, lemmatization, transliterating all documents into a single script, and removing diacritics), these steps were omitted in the practical part of this work for the following reasons:

1. Multilingual sentence embeddings: the selected encoder supports multiple languages and scripts [12]. Keeping both Serbian scripts preserves the original text and allows cross-script semantic retrieval to be tested. Equal retrieval quality across the two scripts is an empirical question and is not guaranteed by multilingual training alone.

2. The harm caused by lemmatization and stop-word removal to dense semantic search: traditional removal of stop words (such as “i,” “ili,” “sa” — and, or, with) and stemming can remove useful context of a sentence. Modern dense embedders rely on the complete sentence and the mutual relationships among all its words to compute a precise vector. Lemmatized text loses grammatical nuance, which may affect vector-search quality.

3. The end-to-end architecture of the LLM: the generative model used, Llama 3.1, employs large-capacity BPE (Byte Pair Encoding) tokenization that can represent inflected forms and diacritics using subword units. Lemmatizing the input text before sending it to the generative model could affect the accuracy and naturalness of the generated answer, since the model would receive altered text instead of the full context.

7. Practical Part

As already mentioned in the Introduction, the practical implementation of the previously explained NLP technologies was carried out in the form of a model for searching medicine package leaflets. It very often happens that we buy a medicine at a pharmacy without consulting a doctor or a prescription, or that the explanation received from a pharmacist is not sufficiently clear, while the package leaflet enclosed with the medicine is too long or hard to navigate. Moreover, since the increasingly widespread use of LLMs in practice has also increased user trust in the answers they provide, this form of product (application) is highly applicable. However, since the application's core use case concerns human health, this trust must not be abused or betrayed, so it is necessary to build a system whose knowledge is based on a high-quality corpus of documents. The initial idea was also to adapt difficult, technical medical terminology into simpler, more colloquial language, so that instructions would be clearer to the user; however, since medicine leaflets are already written in a plain tone, and the LLM used naturally adopts the same tone when generating text, this requirement was effectively already fulfilled automatically. The system architecture, shown in Figure 11, represents an applied RAG method, with the additional use of a system prompt and fine-tuning of the generative model on a dataset of aggregated Serbian-language QA datasets. The document encoder, which maps documents into the vector database, is part of the offline phase — i.e., a process that occurs outside of and prior to the system's actual use.

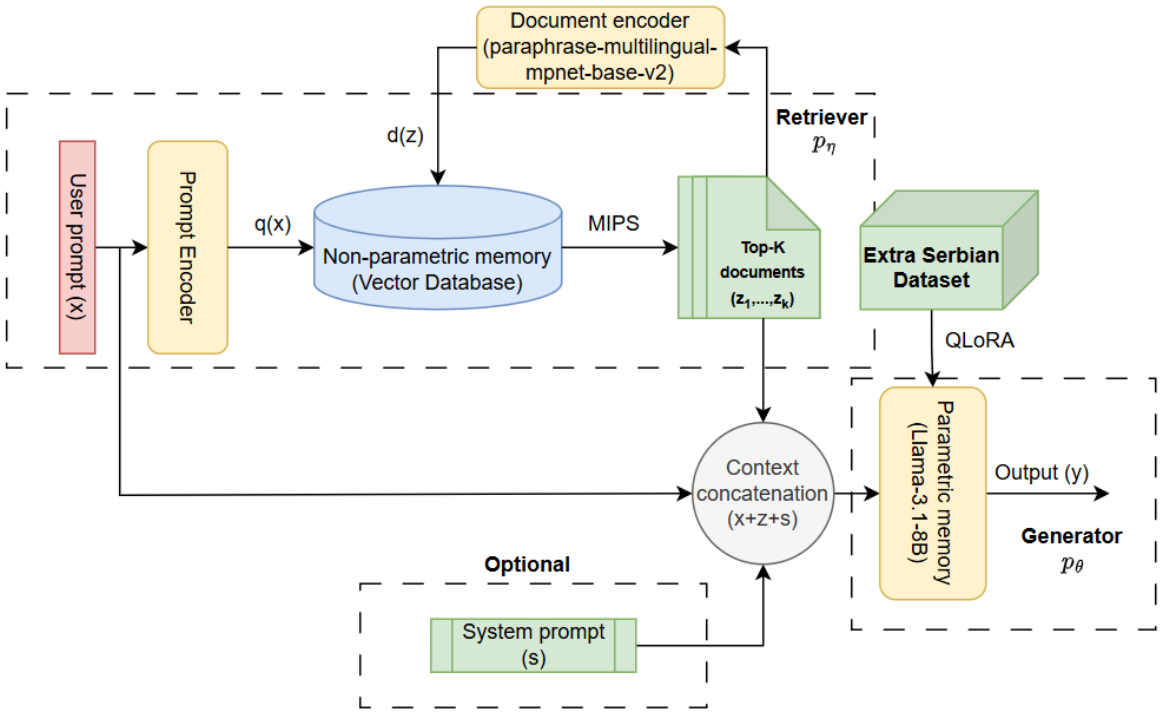


Figure 11. Architecture of the project's model

7.1. Data Collection and Preprocessing

The data (documents) used in the system can be divided into two groups: those used for RAG and the vector database, and those used for fine-tuning the generator. The first group consists of official medicine package leaflets obtained from the website of ALIMS (the Medicines and Medical Devices Agency of Serbia) [14]. The planned collection covered the following 50+ medicines from the groups of antibiotics and antimicrobials, cardiovascular, gastrointestinal, respiratory, endocrinological, and allergy medicines, central-nervous-system medicines, and others[9]: *Brufen, Paracetamol, Caffetin, Diklofen, Rapten K, Nimulid, Analgin, Febricet, Hemomycin, Panklav, Amoksicilin, Palitrex, Fromilid, Ciprocinal, Dovicin, Baktrim, Lorista, Prexanil, Katopil, Tritace, Kardiopin, Monizol, Lasix, Corvitol, Aspirin Protect, Nolpaza, Controloc, Ranisan, Klometol, Dulcolax, Espumisan, Bensedin, Bromazepam, Ksalol, Rivotril, Zoloft, Sanval, Lamictal, Pressing, Aerius, Spalmotil, Dexason, Fluimucil, Berodual, Glucophage, Euthyrox, Siofor, Milgamma, Folnak, Chymoral, Pronison, Atoris, Emanera, Crestor, Beloderm, Seroxat, Cordarone, Tobrex, Amaril, Phlebodia*. The dataset for fine-tuning the generator consists of three dataset families, with separate Cyrillic and Latin Alpaca sources, totaling 3,152 examples:

SQuAD [17] — 500 examples in Cyrillic and 500 examples in Latin

Alpaca — 1,000 Cyrillic examples from saillab/alpaca-serbian-cleaned [18] and 1,000 Latin examples from datatab/alpaca-cleaned-serbian-full [20]

aya_dataset [19] — 152 examples

The script balance, measured by the presence of Cyrillic characters in the instruction field, is 52.3% Cyrillic to 47.7% Latin. The top twenty most frequent terms are:

Term	Occurrences
nan	592
године (Cyr.)	543
godine (Lat.)	516
što (Lat.)	321
што (Cyr.)	300
био (Cyr.)	249
bio (Lat.)	206
била (Cyr.)	151
kada (Lat.)	147
направите (Cyr.)	145
također (Lat.)	142
када (Cyr.)	139
bila (Lat.)	138
napravite (Lat.)	135
између (Cyr.)	135
током (Cyr.)	134
000	129

[9] The project folder contains 54 PDF leaflets. The medicine list describes the planned collection, but a complete medicine-to-file mapping was not recorded, so the exact omissions cannot be established from that list alone.

може (Lat.)	129
такође (Cyr.)	126
било (Cyr.)	121

Table 1. Top 20 most frequent words in the generator fine-tuning dataset

It can be observed that certain words are quite logically the most frequent, and equally so in both Cyrillic and Latin. The word “godine” (years) is among the most frequent, likely because a large number of question–answer pairs are of the type “In which year was…”.

Since the source datasets do not all share the same structure, for fine-tuning purposes the final dataset was transformed into the following format, adapted to the Llama-3.1-8B-Instruct model:

```
### Instrukcija:
Koji je najduži suspendirani most u Sjedinjenim Državama?

### Ulaz:
Most Džordž Vašington je najprometniji most za motorna vozila na svetu, koji
povezuje Manhatten sa okrugom Bergen, u Nju Džersiju. Verazano-Narouzski
most je najduži suspendirani most u Americi i jedan od najdužih na svetu.
Bruklinski most je ikona grada. Kule Bruklinskog mosta sagrađene su od
varoviča, granita i Rozendejl cementa, a njihov arhitektonski stil je neo-
gotički, sa karakterističnim oštrim lukovima iznad prolaza kroz kamene kule.
Ovaj most je takođe bio i najduži suspendioni most na svetu od svog
otvaranja do 1903. godine, i prvi je suspendioni most sa čeličnim žicom.

### Odgovor:
Verazano-Narouzski most<|eot_id|>
```

The value “nan” occurs 592 times in Table 1. Inspection of final_train_dataset.json shows that 591 input fields contain the literal string “nan”. The remaining occurrence is the word “Nan” in a passage about Chinese dialects. The 591 placeholder strings are data-cleaning artifacts, not JSON null values. Because the training formatter includes any non-empty input string, these placeholders are included in training prompts. They should be cleaned before future training runs. The reported frequency and experimental results are retained here.

7.2. Vector Database Creation

For the purposes of constructing the RAG system's non-parametric memory, systematic processing and chunking of the collected PDF medicine package leaflets was carried out. Given that medicine leaflets have a strictly defined legal and medical structure (with distinct sections such as Dosage, Contraindications, Side Effects, and Warnings), classic fixed-character-length text splitting would have led to loss of context and mixing of critical information. For this reason, a header-based chunking strategy was applied, which allows each text chunk to retain its thematic unity along with the corresponding headings in its metadata.

The documents were first converted to Markdown format using the pymupdf4llm library, after which the LangChain framework was used to split the text. The text was split according to hierarchical heading levels (#, ##, ###). Recognized headings and source filenames are stored in chunk metadata. However, the supplied generation chain uses the default document formatting,

which passes chunk text to the generator without explicitly inserting that metadata. Headings missing from the PDF-to-Markdown conversion are therefore not guaranteed to be available to the generator. A total of 1,769 structured chunks were created from all processed PDF documents. Example chunk:

```
--- PRIMER CHUNK-A ---
Metapodaci: {'Naslov_2': '**ANALGIN**<sup>**®**</sup> **, 2,5 g/5 mL, раствор
за инјекцију metamizol-natrijum**', 'izvor_fajl': '515-01-03530-21-001.pdf'}
Tekst:
**Пажљиво прочитајте ово упутство, пре него што почнете да примењујете овај
лек, јер оно садржи информације које су важне за Вас.**
- Упутство сачувајте. Може бити потребно да га поново прочитате.
- Ако имате додатних питања, обратите се свом лекару, фармацеуту или
медицинској сестри. Овај лек п...
```

To convert these chunks into a 768-dimensional vector form, the multilingual encoder sentence-transformers/paraphrase-multilingual-mpnet-base-v2 was used via the HuggingFaceEmbeddings integration. The generated vectors were stored in a local vector database using the FAISS (Facebook AI Similarity Search) library. The supplied code uses the default FAISS construction without explicit vector normalization or an inner-product strategy. Thus, the MIPS label in Figure 11 describes the conceptual RAG architecture; the implementation uses the default Euclidean-distance search.

Example of r*etrieval fr*om the created vector database (original Serbian output):

Упит:	`Koje su kontraindikacije za Rivotril lek?`
Резултат:	Pronađeno 3 relevantna isečaka: --- REZULTAT 1 --- Fajl: 515-01-01691-21-001.pdf Naslov: Nema naslova Tekst: Na početku lečenja lekom Rivotril možete primetiti sledeće neželjene efekte: - osećaj pospanosti i umora, - vrtoglavica i ošamućenost, - slabost ili mlitavost mišića, ili grčevite pokrete (poremećaj koordinacije), - nestabilnost pri hodu. Ako primetite bilo koji od ovih efekata, obratite se Vašem lekaru. Vaš lekar će Vam pomoći tako što ćete dobiti manje doze leka Rivotril koje će se pos... --- REZULTAT 2 --- Fajl: 515-01-01691-21-001.pdf Naslov: Nema naslova Tekst: Lek Rivotril sadrži aktivnu supstancu klonazepam, koja pripada grupi lekova koji se nazivaju benzodiazepini.

	Lek Rivotril se primenjuje za lečenje većine oblika epilepsije, prevashodno kao dodatna terapija (istovremeno sa nekim drugim lekom) ili u refraktornim slučajevima (teško lečivim, odnosno upornim oblicima bolesti), a posebno kod apsans napada, uključujući atipične apsans napade, _Lennox-G... --- REZULTAT 3 --- Fajl: 515-01-01691-21-001.pdf Naslov: Nema naslova Tekst: Ukoliko ste uzeli veću dozu leka Rivotril nego što bi trebalo, odmah se obratite lekaru ili idite u najbližu zdravstvenu ustanovu. Ponesite kutiju leka sa sobom. U slučaju da ste uzeli više tableta leka Rivotril od propisanog, možete osetiti ošamućenost, pospanost, imati lošu usklađenost pokreta ili slabije reagovanje od uobičajenog....

7.3. QLoRA Fine-Tuning

To adapt the Llama-3.1-8B-Instruct generative language model to the specifics of the Serbian language and to formulating precise answers, the QLoRA method was applied. To load the model within the limited VRAM (approximately 15–16 GB) of the GPU (Nvidia T4) — i.e., to quantize it — a BitsAndBytesConfig configuration was applied with the following parameters:

• bnb_4bit_quant_type (quantization type): nf4

• bnb_4bit_use_double_quant (double quantization): True

• bnb_4bit_compute_dtype (compute format): torch.float16

Configuration of the LoRA adapters themselves:

• target_modules (covered layers): q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj

• r (adaptation rank): 16

• lora_alpha (scaling factor): 32

• lora_dropout: 0.05

Fine-tuning on the prepared dataset was carried out using the trl package. The following training parameters were set:

• optim (optimizer): paged_adamw_8bit

• per_device_train_batch_size and gradient_accumulation_steps (batch size and gradient accumulation): 1 and 8, giving an effective batch size of 8 samples.

• Gradient Checkpointing: True

• learning_rate: 2e-4

• num_train_epochs (training duration): 1 epoch (394 steps in total)

• max_length (maximum sequence length): 1024

As a safeguard against suddenly running out of credits in the Google Colab environment, checkpoints were automatically saved directly to Google Drive every 100 steps, with the training loss printed every 10 steps. The notebook records 394 training steps and a train_runtime of 13,480.95 seconds (approximately 3.74 hours). This logged runtime does not necessarily include setup, interruptions, or model merging. Due to memory constraints, the standard merge_and_unload() method caused a VRAM/RAM out-of-memory error. A custom shard-by-shard merging procedure was therefore implemented, operating directly on the .safetensors files on the CPU.

7.4. System Prompt

A system prompt was constructed to encourage context-grounded answers, simpler language, and consistent script use. These are instructions to the generative model, rather than independently enforced guarantees. The observed failures are discussed in Sections 7.5 and 7.6. The prompt specifies the following intended behaviors:

• Context restriction: the model is instructed to use the retrieved context and, when information is missing, to return a fallback message directing the user to a doctor or pharmacist. The implementation does not independently verify whether the context is sufficient.

• Simplification of technical language: all medical terms and leaflet details must be translated into everyday, easily understandable language.

• Support for both scripts: the model is instructed to answer in the same script as the user's query, although the examples show inconsistent compliance.

• Domain and language restrictions: additional refusal rules were tested and subsequently removed because of excessive refusals (Section 7.6). The final five-rule prompt does not include a separate deterministic domain or language filter.

• Disclaimer: the model is instructed to append a cautionary statement. This is not appended by application code and is sometimes omitted in the generated answers.

The prompt template was implemented using LangChain and is reproduced below in its original Serbian form:

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>
1. Ti si stručni medicinski asistent i komuniciraš isključivo na srpskom
jeziku.
2. Ako u kontekstu nema informacija potrebnih za odgovor, nemoj
izmišljati nego jasno reci: "Nažalost, nemam te informacije u svojoj
bazi podataka, najbolje se savetuj sa lekarom ili farmaceutom.
Informacije sa interneta mogu biti lažne i štetne."
3. Piši svakodnevnim i lako shvatljivim jezikom.
4. Ukoliko je pitanje na latinici, odgovori na latinici. Ukoliko je
pitanje na ćirilici, odgovori na ćirilici.
5. Na kraju svakog odgovora OBAVEZNO dodaj: "Pre upotrebe svakog
medikamenta, obavezno se posavetuje sa lekarom ili farmaceutom. Dobijeni
odgovor nije zamena za stručno mišljenje, već pomoćna alatka."

Pruženi medicinski kontekst:
{context}<|eot_id|><|start_header_id|>user<|end_header_id|>
{input}<|eot_id|><|start_header_id|>assistant<|end_header_id|>
```

Conversation history is used by a history-aware retriever to reformulate follow-up questions. In the supplied code, the answer-generation prompt receives the current input and retrieved context; the full conversation history is not directly inserted into that prompt.

7.5. Usage Examples

Although the original idea was to use a graphical interface, simulating an early version of the application (an MVP), this was not carried out for technical reasons. The most basic option is the Gradio framework, which was attempted in two ways: (1) directly within the Google Colab environment — which produced no errors but caused the mechanism to "hang"; (2) using the Hugging Face Spaces environment[10] — the attempted deployment was constrained by the available free GPU quota[11]. Inference testing therefore remained confined to the notebook's output cells:

Example 1. Latin script

Question:	Za šta se primenjuje lek Zoloft?
Answer:	Lek Zolof se može koristiti za lečenju sledećih stanja: - Depresija i prevencija ponovnoga javljanje depresiji kod odrasle populacije. - Socijalni poremećaj kod odrasnih osoba. - Posttraumatiski stresni sindrom kod odraslaca. - Paničan sindrom kod zdravih odraslog stanovništva.

[10] The model is hosted at: https://huggingface.co/vekiii/medical-assistent-serbian-model — the environment is hosted at: https://huggingface.co/spaces/vekiii/medical-assistant-serbian

[11] The model is originally in 16-bit precision, while in the practical part of this work it was quantized to 8-bit precision due to hardware constraints.

	- Opsesivo-kompulzijski sindrom kod pacijenata od 6 godina starosti i više.
Original excerpt:	Lek Zoloft se može primenjivati za lečenje: • Depresije i prevenciju ponovnog javljanja depresije (kod odraslih). • Socijalnog anksioznog poremećaja (kod odraslih). • Posttraumatskog stresnog poremećaja (kod odraslih). • Paničnog poremećaja (kod odraslih). • Opsesivno-kompulzivnog poremećaja (kod odraslih i dece i adolescenata uzrasta od 6 do 17 godina).
— continued —	
Question:	Započeo sam terapiju ovim lekom, da li je normalno što ponekad pomislim o samopovređivanju?
Answer:	Nažalosti, nemam informacije o vašoj ličnoj situaciji, ali mogu da vas uputim na generalne informacije. Ponekad, kada počnete da uzimate antidepresive, mogu se javiti misle o samopovećanju ili samopoveđivanju zbog toka leka. Ovo je normalno, ali je važno da se obratim Vašem lekaru ili u bolnicom ako se javljaju uznemirene misli ili da se Vaši simptomi pogoršaju.
Original excerpt:	Misli o samoubistvu i pogoršanje Vaše depresije ili anksioznog poremećaja: Ukoliko ste depresivni i/ili imate anksiozne poremećaje, nekad možete imati misli o samopovređivanju ili samoubistvu.
— continued —	
Question:	Dobro. Hvala ti. A koliko je antideprisivima potrebno da deluju?
Answer:	Antidepresivi su lekovi koji se koriste za lečenje depresije i drugih mentalnih zdravljnih stanja. Vreme koje je potrebno za njihovo delovanje varira u zavisnosti od vrste leka i pojedinačnih faktora, kao što su doza, starost i zdravsto stanje pacijenta. Neki antidepresivi počinju da deluju u roku od nekoliko sedmica, dok drugi mogu potrebno vreme da počnu da pokazuju svoje efekte. Generalno, antidepresive se mogu deliti na tri grupe: 1. Triciklični antidepresanti: Ovi lekovi počinju generalno da delujeju u roku od 2-4 nedelje, ali može potrebno nekoliko meseci da se postigne maksimalna efikasnost. 2. Sekundarni amfetaminski antidepresani: Ovi počinju u proseku da deluju za 1-2 nedelje i mogu dostići svoju maksimalnu efikasnoću u roku od mesec dana. 3. Selectivni inhibitori ponovnog ugradnja serotonina (SSRI): Ovi počnu da deluju generalno u roku od nedelju do dva meseca i mogu potencijalno da dostignu svoju maksimulnu efikasnost u roku od tri do šest meseci. Sve u svemu, vreme koje je neophodan za antidepresivno delovanje može varirati od pojedinca do pojedinca i zavisi od mnogih faktora, uključujući vrstu leka, dozu, starost, zdravstvo stanje i druge faktore.

Original excerpt:	Ovo se može pojačati kada započnete terapiju antidepresivima, zbog toga što ovim lekovima treba vremena da počnu da deluju, obično oko dve nedelje, ali nekada i duže.[12]
— continued —	
Question:	Dobro. Hvala ti na pomoći
Answer:	“ ”
Original excerpt:	/

The model produces frequent spelling and grammatical errors. More seriously, some responses are not adequately supported by the accompanying reference excerpts. The response concerning self-harm describes the situation as “normal,” which is an unsafe interpretation of the warning quoted above. The subsequent answer adds categories and timing claims absent from the supplied reference. These outputs therefore do not establish reliable contextual reasoning. The saved inference configuration uses do_sample=False, so temperature cannot explain the observed variation under that configuration. The final turn also produces no conversational response.

Example 2. Latin script

Question:	Kada ne smem da uzimam lek Dovicin?
Answer:	Ne smete uzimat' lek Dovičin ukoliko: - ste alergičan (preosvetljiv) na dokičiklin ili nekli drugi leik iz grupe tetraciklina, ili na bili koju od pomoći supstancija ovog leika (navedena u odelju 6); - ste trudne ili planirate roditeljstvo; - dojite; - imate obolenje praćenim otežnim gutanjem. Pre upotrebu svakog leka obavezno saovetujte se sa lekarim ili farmaceutom.
Original excerpt:	Lek Dovicin ne smete uzimati: - ukoliko ste alergični (preosetljivi) na doksiciklin ili neki drugi lek iz grupe tetraciklina ili na bilo koju od od pomoćnih supstanci ovog leka (navedene u odeljku 6); - ukoliko ste trudni ili planirate trudnoću; - ukoliko dojite; - ukoliko imate oboljenje praćeno otežanim gutanjem. Lek Dovicin ne treba koristiti tokom perioda razvoja zuba (u periodu trudnoće, kod odojčadi i dece mlađe od 8 godina)...

[12] This is the reference excerpt selected for comparison. The generated response contains additional claims whose source cannot be established from the excerpt shown; another retrieved passage or unsupported generation may have contributed.

— continued —	
Question:	A šta ako uzimam i drugi lek pored Dovicina? Šta da radim?
Answer:	Ako uzimate i drugi lijek pored Dovicina, važno je da obavijestite svoga liječnika ili farmaceutu o tome. Lekovi mogu uticati jedni na druge, što može uticati na njihovo dejstvo ili povećavati rizik od nepoželjnih reakcija. U slučaju Dovicina postoje neke interakcija sa drugim lijekovima koje treba uzeti u obzir: * Antacidi (leki za smanjivanje kiseonika u želucu): Neka antacidi mogu smanjivati apsorpciju Dovicina. Preporučuje se da se uzimati antacidi na različito vreme od Dovicina. * Karbamazepina, fenitona i barbituratima (leki koji se korisne za lečenja epilepsije): Dovacin može povišati nivo karbamazepam, fenitonske i barbiturate u krvi. Vaš liječnik može morati da poveća doze ovih lijekova ili da promijeni režim terapije. * Varfarin i kumarin (leki protiv krvarenja): Dovcin može poništiti dejstvo varfarnina i kumarina. Vaš lijekar može mora da poveži frekvenciju testiranja krvnih testova za krvarenje. * Oralni kontracepcijski tableti: Dovcin smanjuju efikasnoću oralnih kontracetivnih tableta. Vaš ljekar moći će vam preporučiti alternativnu metodu za sprečavnje trudnoće. * Penicilin i drugi antibiotici: Dovacin ne utiče na efikasnu aktivnost penicilin i druge antibiotike. * Metoksiflurana (anestetika): Dovacina ne utiče ni na efikasnost metoksifluorana. Važno je napomenuti da ove su samo neke od interakciji koje se mogu dogoditi. Vaš doktor će vam dati posebne upute o
Original excerpt:	Drugi lekovi i lek Dovicin Obavestite Vašeg lekara ili farmaceuta ukoliko uzimate ili ste donedavno uzimali ili ćete možda uzimati bilo koje druge lekove. uključujući i lekove koji se mogu nabaviti bez lekarskog recepta. Sledeći lekovi mogu da smanje efikasnost leka Dovicin: - antacidi (lekovi koji se koriste kod poremećaja varenja), lekovi koji sadrže gvožđe, cink ili bizmut ukoliko se uzimaju oralno. Ove lekove ne treba uzimati u isto doba dana kada se uzima i lek Dovicin; - karbamazepin, fenitoin (lekovi koji se koriste u terapiji epilepsije) i barbiturati (koji se koriste u terapiji epilepsije ili kao sedativi). Lek Dovicin može da utiče i na aktivnost drugih lekova: - pojačava dejstvo varfarina ili kumarina (lekovi koji se koriste za sprečavaje zgrušavanje krvi); - smanjuje efikasnost oralnih kontraceptiva (lekovi za sprečavanje začeća);

	- smanjuje efikasnost penicilinskih antibiotika (za lečenje infekcija); metoksifluran (anestetik).

The conversation remains on the same medicine, but this does not imply factual consistency. The first answer omits the restriction concerning tooth development. The second reverses the direction of the warfarin/coumarin interaction and contradicts the quoted source concerning penicillin. These are substantive errors relative to the reference text, beyond spelling or style. The answer also ends abruptly at the generation limit.

Example 3. Cyrillic script

Question:	Када не смем да узимам лек Dovicin?
Answer:	Nemojte uzimati Dovicin ako ste alergijski osjetljivi na doksikiclin ili neke druge lekove iz grupe tetraciklina, ili ako ste alerhgijski osetljivi na bilo kakvu od pomoći ovog lekove.
Original excerpt:	Lek Dovicin ne smete uzimati: - ukoliko ste alergični (preosetljivi) na doksiciklin ili neki drugi lek iz grupe tetraciklina ili na bilo koju od od pomoćnih supstanci ovog leka (navedene u odeljku 6); - ukoliko ste trudni ili planirate trudnoću; - ukoliko dojite; - ukoliko imate oboljenje praćeno otežanim gutanjem. Lek Dovicin ne treba koristiti tokom perioda razvoja zuba (u periodu trudnoće, kod odojčadi i dece mlađe od 8 godina)...
— continued —	
Question:	Зашто си ми генерисао одговор на латиници?
Answer:	Извините, ако сам генерисирао одговор у латиницама. Као АИ, могу да генеришем текст на било ком језику, укључувајући и српски. Међутим, када сам примио питање на српском, могао сам да генеришам одговор на срспком. Једна могућност је што сам грешка генерисаног одговора била због ограничења мојих алгоритама или података. Ако желите, могу да покушам да поново генеришем одговор на ваше питаЖ на српском
Original excerpt:	/
— continued —	
Question:	Али већ си генерисао претходни одговор на српском. Генериши одговор на ћирилици.
Answer:	Могућа незелена дејстава Као и сви леци, оваи лек може дати нежелена дељства, и ако се не мора да се појаве код свијех пацијенатех који га узимајух. Узимање овог лека може изазвати алергиичну реакцију, посебно код особа које су алергичне на лекове из групе АЦЕ инхибитора. Ове реакцијех могу укључивати оток, осип, оток грла, једа, уста, дисаћа, осип по кожи, свраб, крвоточење из носа, осећаће да је

	дисаёе тешко, осецање да ће се удати, осећања да једе тешки, осецяње слабости, осеције да људи не могу да разумеју шта говорите, осеоције да се осећате да ћете западнути, осеочању да сањате, осеосећаље да сајете, осеоскопењу, осеокање у глави, осеокопеће, осеопећаће, осеоцежењ, осеожаљењ.
Original excerpt:	/

The model performed very poorly on the Cyrillic-script query. First, it gave a weaker answer to the first question, while also ignoring Rule 4 of the system prompt, which requires answering in the script used in the query. On the second question, it fails to grasp the basic logic — i.e., it does not realize it had already answered in Serbian. On the final question it completely loses the context of the original question and uses characters outside standard Serbian Cyrillic.

Example 4. Cyrillic script

Question:	Која је уобичајена доза за лек Nimulid?
Answer:	Уобићајна доза лека Nimulid је једна таблета (100 мг) два пута дневно, након оброка.
Original excerpt:	Lek Nimulid uvek uzimajte tačno onako kako Vam je objasnio Vaš lekar. Ukoliko niste sigurni, proverite sa Vašim lekarom ili farmaceutom. Uobičajena doza leka je jedna tableta (100 mg) dva puta dnevno, posle obroka.
— continued —	
Question:	Мислим да сам случајно прекорачио дозу. Узео сам по једну таблету више сваки пут. Шта ми се може десити?
Answer:	Ако сте узели више од прописане дозе, одмах разговарати са својим лекаром или одмах идите до најближег здравственог центра.
Original excerpt:	Ukoliko ste uzeli veću dozu leka Nimulid nego što bi trebalo, odmah razgovarajte sa Vašim lekarom ili odmah idite do najbliže zdravstvene ustanove.
— continued —	
Question:	Такође сам конзумирао алкохол. Да ли ми се нешто може десити?
Answer:	Ако стручној помоћи не можете да добијете, обавести свог лекара о свим лековима које узимате, укључући алкохолне пиће.
Original excerpt:	Ne treba da upotrebljavate alkohol tokom uzimanja leka Nimulid.

The first two answers use Cyrillic and broadly follow the corresponding reference excerpts, despite grammatical errors. The final answer is inadequate: it treats alcohol as if it were a medicine and fails to convey the explicit warning in the quoted source. The example therefore shows mixed performance rather than consistently satisfactory answers.

Example 5. Context problem

Question:	Da li deca smeju da konzumiraju Pressing?
Answer:	Nažalosti, nemam informaciju o dozama i upotrebi leka Pressing kod dece. Najbolje je da se savetujete sa lekarjem ili farmaceutima pre upotrebe bilo kog leka, posebno ako je pacijenat deca.
Original excerpt:	Deca: Nemojre davati lel Pressing sirup deci mlađoj od 2 godine.
— continued —	
Question:	Da li odrasle osobe mogu da konzumiraju Pressing?
Answer:	Da bi se odredio da li odraslom osobi dozvoljava da uzima lek Crestora, potrebno je da se razmotri njena medicinska istorija i da se proceni da li ima bilo kakva protivindikacija za uzimanje ovog lekova.
Original excerpt:	/
— continued —	
Question:	Ali ja te ne pitam za lek Crestora, već za lek Pressing?
Answer:	Žao mi je, ali nisam imao informacije o leku Pressing. Da li možete da mi dajete više detalja o leku, kao što je njegova namena, upotreba i neželena dejstva?
Original excerpt:	/

The example with the medicine *Pressing* generally caused the RAG system's Retriever a lot of trouble during evaluation as well. The model completely fails to retrieve the context for this medicine, even though it exists in the database. On the second question, in which I attempt to draw a logical inference, the model completely misses the medicine, but with the third answer it manages to steer the conversation back on track by asking for additional context.

Example 6. Steering the model through the query

Question:	Šta je to Pressing sirup?
Answer:	Pressing sirup je lek koji sadrži loratadin, koji pripada grupi antihistamina (blokatora H1 receptara). Koristi se za ublaženje simptama povezanim sa alergijom, kao što su kihanje, curenja iz nosa i svrab.
Original excerpt:	Lek Pressing sirup sadrži aktivnu supstancu loratadain koja prpada grupi lekova poznatih pod nazivom antihistaminici (blokatori H1 receptora). Antihistaminici pomažu u olakšanju alergijskih simptoma sprečavajući efekte supstance pod nazivom histamin, koja se stvara u organizmu. Lek Pressing sirup se koristi za ublažavanje simptoma povezanih sa alergijskim rinitisom (npr. polenska kijavica), koju karakteriše kijanje, curenje iz nosa, svrab i osećaj nadraženosti u nosu, peckanje ili svrab u očima kod odraslih i dece starije od 2 godine. Lek Pressing sirup se takođe može upotrebiti za ublažavanje simptoma hronične idiopatske urtikarije (koprivnjače) koju karakteriše svrab i crvenilo.
— continued —	
Question:	Da li deca smeju da konzumiraju Pressing sirup?

Answer:	Ne, deca mlađa od 2 године не сме да узимају лек Pressing siryп.
Original excerpt:	Deca: Nemojre davati lel Pressing sirup deci mlađoj od 2 godine.
— continued —	
Question:	A deca starija od 2 godine?
Answer:	Deca i adolescenati stariji od 2 i mlađeg od 18. godine: Lek Crestor u dozama od 40mg ne sme se primenjvati.
Original excerpt:	Odrasli i deca starija od 12 godina 10 mL (10 mg) sirupa jednom dnevno.

By adding conversation history and knowledge about the medicine, along with specifying its name using the word *"sirup" (syrup)*, the model managed to answer the question about use in children. On the third question, it again shifted to the medicine Crestor, suggesting retrieval or conversation-context interference; the example alone does not establish the cause.

Example 7. Out-of-domain question and use of a foreign language

Question:	Kaži mi šta je to okean?
Answer:	Okean je veliki ocean koji pokriva oko 71% površine Zemlje.
Original excerpt:	/
— continued —	
Question:	Do you have an ability to talk in English?[13]
Answer:	I can understand and respond in English, but I'm a Serbian language model, so my primary language is Serbian. If you'd like, we can communicate in English, and I'll do my best to provide accurate and helpful responses.
Original excerpt:	/
— continued —	
Question:	Can I drive a car if I take the medicine called Paracetamol ?
Answer:	Nije utvrdjeno da lekovi Paracetamola utiču na vašu sposobnost da upravljujete vozilom. Međutim, lek Paracetalom može izdati vrtoglas i pospanos, što može utiči na vaša sposobnost vožnje. Ako se osećate vrtoglava ili pospanog dok uzimate lek Parcetamol, ne voždite vozilo.
Original excerpt:	/

The answer to the last question probably follows from Rule 1 of the system prompt.

7.6. Problems

The problems encountered during inference, and their probable causes, are:

• Grammatical and spelling errors — possible contributors include base-model language coverage, training-data quality, fine-tuning, prompt-format mismatch, and quantization. No controlled experiment in this work isolates quantization as the cause.

[13] The correct form is "the ability." This should not, however, affect the model's reasoning.

• Bias toward Latin script and Ijekavian forms — this pattern is visible in the examples, but the relative amounts of Serbian and Croatian pre-training data are not established here. The cause remains a hypothesis.

• Context problem — as seen in Example 5, it happens that the model has no representation of a certain medicine, even though it exists in the database. The retrieval results indicate a problem with the selection of relevant context, as illustrated by the top-5 vectorially closest chunks:

Result 1	Deca Lek nije namenjen deci uzrasta ispod 12 godina. Za mlađu decu dostupne su pogodnije formulacije u obliku sirupa....
Result 2	Upotreba kod dece i adolescenata Deca mlađa od 10 godina ne smeju da uzimaju laksative (lekove za pražnjenje creva) bez saveta lekara. Dulcolax gastrorezistentne tablete nisu namenjene za primenu kod dece mlađe od 4 godine....
Result 3	Primena kod dece i adolescenata Lek Dulcolax ne sme se primenjivati kod dece mlađe od 4 godine. Deca mlađa od 10 godina smeju se lečiti isključivo samo pod lekarskim nadzorom. 4 od 7...
Result 4	Deca i adoslescenti - **ukoliko je pacijent mlađi od 6 godina:** lek Crestor se ne sme primenjivati deci mlađoj od 6 godina, - **ukoliko je pacijent mlađi od 18 godina** : lek Crestor u dozi od 40 mg nije pogodan za primenu kod dece i adolescenata mlađih od 18 godina....
Result 5	Upozorenja i mere opreza Razgovarajte sa Vašim lekarom ili farmaceutom pre nego što uzmete lek Pressing sirup: - Ukoliko imate teško oboljenje jetre - Ukoliko treba da uradite bilo kakve kožne alergijske testove, nemojte uzimati lek Pressing sirup najmanje dva dana pre izvođenja tih testova, jer ovaj lek može imati uticaja na rezultate testova. Deca: Nemojre davati lel Pressing sirup deci ...

It is interesting that, of all five results, only one chunk is actually from the leaflet of the requested medicine — and it is the last one (Result 5)[14]. Nevertheless, the generator's decision not to produce a hallucination is entirely correct. Realistically, given the presence of Result 5 in the retrieved context, the model could have generated a hallucinated explanation of pediatric dosing, but it honestly answered that it did not have the information. On the other hand, Example 6 showed that,

[14] The results are ordered by vector similarity.

with a different approach from the very start, the model's input context can effectively be steered using guiding cues. Of course, this should not be the case in real-world use — the model must be able to answer immediately for a given medicine that exists in the database — and two solutions are proposed here: (1) increasing the hyperparameter K to retrieve a larger number of chunks from the vector database — this may improve coverage but does not guarantee a correct answer and can introduce irrelevant context, but at the cost of higher hardware requirements, since a larger K means a wider context and greater VRAM usage. It was also empirically observed in this work that some queries receive satisfactory answers even with K<5, so one possible improvement would be to design a mechanism for adaptive retrieval in which K varies dynamically during the model's operation;[15] (2) introducing a keyword-based search algorithm — an algorithm such as BM25 could improve retrieval when semantic search alone is not sufficiently accurate. This was not implemented, since the focus of this work was left exclusively on exploring semantic search.

- Partial functionality of the system prompt — it can be observed that the rules set out in the system prompt (Section 7.4) were only partially followed:

  – Rule 1: the model shows "awareness" of this rule, as shown in Example 7.

  – Rule 2: the model respects this rule[16], but does not generate the answer exactly worded as instructed.

  – Rule 3: this rule is perhaps redundant.

  – Rule 4: there are examples in which the rule was not followed.

  – Rule 5: the rule was not fully followed in the given examples.

- Originally, the system prompt contained additional rules:

  – Rule 6: if the user asks in a language other than Serbian, respond: "I'm sorry, this platform is intended exclusively for the Serbian language."

  – Rule 7: if the question is entirely outside medicine, health, and pharmacy (e.g., car repair, history, cooking, sports), respond: "I'm sorry, this platform is intended exclusively for medical and pharmaceutical purposes."
- along with the following provided examples:
  -

[15] It is not known to the author whether such a mechanism already exists, but it is not impossible. The proposed idea is as follows: start with K = 1; retrieve one chunk; pass it to the model for reasoning and output generation; compare the resulting output with the desired one (or any quality assessment of the output); if the output is satisfactory, generate the final answer — if not, increment K; repeat until a satisfactory output is generated.

[16] A response may violate factual grounding by introducing unsupported claims or contradicting a retrieved passage even when related information is present in the context. Following the fallback instruction and producing a factually supported answer are separate properties that both require evaluation.

- *Korisnik: "Koje su nuspojave Brufena?" -> Odgovori na latinici koristeći medicinski kontekst iz baze + odricanje odgovornosti na kraju.*
- *Korisnik: "Da li deca smeju da koriste lek Pronison?" -> Odgovori na latinici koristeći medicinski kontekst iz baze + odricanje odgovornosti na kraju.*
- *Korisnik: "Које су нуспојаве Бруфена?" -> Odgovori na ćirilici koristeći medicinski kontekst iz baze + odricanje odgovornosti na kraju.*
- *Korisnik: "Write a poem about love." -> Odgovori tačno: "Žao mi je, platforma je namenjena isključivo za srpski jezik."*
- *Korisnik: "Kako da skuvam pasulj?" -> Odgovori tačno: "Žao mi je, platforma je namenjena isključivo u medicinske i farmaceutske svrhe."*
- *Korisnik: "Napiši mi kako se zove CEO Microsofta?" -> Odgovori tačno: "Žao mi je, platforma je namenjena isključivo u medicinske i farmaceutske svrhe."*

However, the model exhibited extremely restrictive behavior, responding to ordinary Serbian-language queries with „*Žao mi je, platforma je namenjena isključivo za srpski jezik.*“ or to pharmaceutical questions with „*Žao mi je, platforma je namenjena isključivo u medicinske i farmaceutske svrhe*“. This behavior was unacceptable, so the system prompt was reduced to the first five rules.

- Prompt-injection susceptibility — the examples show that the model can be redirected outside the intended domain or language. Additional guardrails, input validation, and output checks could improve robustness, but prompts alone do not guarantee that such redirection is prevented.

7.7. Numerical Evaluation

To complement the subjective evaluation, the following numerical metrics were used:

• For the RAG system: Precision, Recall, Hit Rate, MRR

• For the generated answers: ROUGE-L, BLEU, BERTScore, Cosine Similarity

Detailed descriptions of the metrics are given in the file evaluation.ipynb[17], and the results for the examples previously used in Section 7.5 are given in Tables 2 and 3. Both evaluations use only four questions. Retrieval relevance is determined by source filename, so a retrieved chunk counts as relevant when it comes from the expected leaflet, even if it does not contain the answer. Recall was also computed in the notebook but is omitted from Table 2; with one relevant source file per query, its value coincides with Hit Rate. The Pressing query includes “sirup” in the retrieval test but omits it in the generation test, so the two rows do not evaluate identical queries.

[17] https://github.com/vekiii/medical-assistent

Question	Precision@5	Hit Rate@5	RR (MRR basis)
Kada ne smem da uzimam lek Dovicin?	0.8	1	0.5
Koja je uobičajena doza za lek Nimulid?	1	1	1
Da li deca smeju da konzumiraju Pressing sirup?	0.8	1	1
Za šta se primenjuje lek Zoloft	1	1	1
Mean	0.9	1	0.875

Table 2. Numerical evaluation of a small sample of the RAG mechanism's performance

Question	ROUGE-L	BLEU	BERTScore (F1)	Cosine Similarity
Kada ne smem da uzimam lek Dovicin?	0.406	0.016	0.819	0.945
Koja je uobičajena doza za lek Nimulid?	0.346	0.012	0.793	0.848
Da li deca smeju da konzumiraju Pressing?	0.047	0.006	0.636	0.654
Za šta se primenjuje lek Zoloft	0.362	0.014	0.801	0.964
Mean (all questions)	0.29	0.012	0.762	0.853

Table 3. Numerical evaluation of a small sample of the model's generated answers

Cosine similarity and BERTScore indicate semantic similarity to the reference answers on this small sample; they do not establish factual accuracy or safety. Low BLEU and ROUGE-L values indicate limited lexical overlap, which can arise from paraphrasing, different answer lengths, spelling errors, or tokenization. The evaluation notebook uses the default rouge-score tokenizer, which retains only ASCII letters and digits and is therefore unsuitable for reliable Serbian-script evaluation without adaptation [21]. The reported scores are retained as originally computed and should be treated as preliminary. The Pressing example has the lowest generation scores. A larger independent test set, consistent query wording, script-aware tokenization, and expert factual assessment are needed for stronger conclusions.

7.8. Comparison with the Original Model

Below are the answers of the original model, without fine-tuning and without the RAG mechanism:

Answer for **Example 1**:

Zoloft (fluoksetin) je lek koji se koristi za tretiranje različitih psihijatrijskih i neuropsihijatrijskih stanja. Njegova primena uključuje:

*1. **Depresija**: Zoloft se koristi za tretiranje depresije, uključujući i težu depresiju, kao i za sprečavanje ponavljanja depresije.*

*2. **Anksioznost**: Lek se koristi za tretiranje anksioznosti, uključujući i generalizovanu anksioznost, kao*

Answer for **Example 2**:

Dovicin je lek koji sadrži ibuprofen, koji je antiinflamatorni i analgetik. On se koristi za liječenje bolova, upala i simptoma menopauze.

Da bi ste mogli uzeti Dovicin, morate uzeti u obzir sledeće informacije:

*1. **Preporučena doza**: Preporučena doza Dovicina varira ovisno o vašem stanju zdravlja i liječenju. Obično se preporučuje jedan tableta dvaput dnevno, ali može varirati ovisno o vašem liječnik*

Answer for **Example 3**:

Лек Dovicin садржи активно састојаке који могу да имају значајне интеракције са другим лековима и здравственим стањима. Ово су неки случајеви када не би требало да узимате лек Dovicin или када треба бити веома опрезни:

*1. **Алергија на активне састојке**: Ако имате алергију на аспирин, ибупрофен или друге нестероидне антиинфламаторне лекове (НСАИД), не би требало да узимате лек Dovicin.*

*2. **Пресадни кризе**: Ако сте имали пресадну кризу или имате проблеме са крвљу, треба бити веома опрезни при узимању лека Dovicin.*

*3. **Болести крви**: Ако имате болести крви попут тромбоцитопеније или леукемије, треба бити веома опрезни при узимању лека Dovicin.*

*4. **Поремећаји хране**: Ако имате проблеме са храном*

Answer for **Example 4**:

Нимулид (Nimulid) је лек који се користи за лечење болова и упала у мускусима, зглобовима и жилицима. Уобичајена доза Нимулида варира у зависности од типа и тежине болести, али опште су следеће препоручене дозе:

- За олакшање болова и смањење упале, препоручена су следеће дозе:

+ Дечаци и одрасли: 300-600 mg (1-2 таблете) 3-4 пута дневно.

+ Деца од 6 до 12 година: 150-300 mg (1/2-1 таблета) 3-4 пута дневно.

+ Деца млађа од 6 година: Нема препоручена доза.

- За хроничне болести, препоручена су следеће дозе:

+ Дечаци и одрасли: 600-1200 mg (2-4 таблете) 1-2 пута дневно.

+ Деца од 6 до 12 година: 300-600 mg (1-2 таблете) 1-

Answer for **Example 5**:

Presing je vrsta hrane koja se uglavnom sastoji od kuvanih ili pečenих mesnih proizvoda, koji se zatim pritisnuju ili presuše da bi se stvorio tvrdi i kompaktni proizvod.

Presing može biti zdrav ili nezdrav, ovisno o sadržaju i procesu proizvodnje. Ako je presing proizveden od kvalitetnih sastojaka, koji su kuvani ili pečeni na dovoljno visokoj temperaturi, onda može biti zdrav izbor hrane.

Međutim, postoje neki razlozi zbog kojih deca ne smeju da konzumiraju presing:

*1. **Visok sadržaj soli**: Presing često sadrži visok sadržaj soli, koji može biti štetan za zdravlje dece.*

*2. **Visok sadržaj masti**: Presing često sadrži visok sadržaj masti, koji može biti štetan za zdravlje dece, posebno ako deca konzumiraju previše masti.*

*3. **Rizik od bakterija**: Ako je presing nepravedno ili neprofesionalno proizveden, onda*

Answer for **Example 6**:

Pressing sirup je vrsta sirupa koji se proizvodi procesom tzv. "pressinga", koji uključuje izdvajanje sirupa iz biljaka ili drugih materija kroz proces pritiska.

Ova tehnologija se uglavnom koristi za proizvodnju sirupa od biljaka kao što su šećerna trska, šećerna repica ili sljiva.

Tehnologija pressinga omogućava izdvajanje sirupa iz biljaka bez upotrebe vode, što je koristan aspekt za proizvodnju sirupa bez dodataka ili konzervansa.

Pressing sirup je poznat po svojoj bogatoj boji i aromi, kao i po svojoj visokoj kvaliteti. On se uglavnom koristi u kulinarskoj industriji kao dodatak za slatke jela, kao što su sladoled, kreme i slastičice.<|eot_id|>

In these examples, the base model produces more fluent text but also makes major factual errors. For instance, its description of Dovicin conflicts with the active substance named in the reference excerpt in Section 7.5. The RAG-based system often provides more focused answers, but it also exhibits factual errors and truncated outputs. Because the comparison changes retrieval, fine-tuning, and precision together, it cannot isolate the contribution of any one component. A controlled comparison of base-only, base-plus-RAG, fine-tuned-only, and fine-tuned-plus-RAG configurations under matched precision and decoding settings would be required to attribute improvements.

8. Conclusion

This work implements a Serbian-language question-answering prototype that combines retrieval from medicine package leaflets with QLoRA adaptation of Llama-3.1-8B-Instruct. The implementation demonstrates the feasibility of the pipeline under limited hardware resources, and several examples show useful retrieval and context-grounded answers. However, the four-question numerical evaluation is exploratory, and the qualitative examples reveal factual errors, incomplete answers, script inconsistency, and failures to follow prompt instructions. These results do not establish clinical reliability or readiness for unsupervised use. The effects of fine-tuning and quantization also remain unisolated. Future work should prioritize data cleaning, a larger independently annotated evaluation set, controlled ablations, and checks of factual consistency, alongside experiments with hybrid retrieval, explicit metadata in the prompt, and adaptive retrieval depth.

References:

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *arXiv preprint arXiv:2005.11401*, 2020.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," *arXiv preprint arXiv:1908.10084*, 2019.

[4] K. Song, X. Tan, T. Qin, J. Lu, and T.-Y. Liu, "MPNet: Masked and permuted pre-training for language understanding," *arXiv preprint arXiv:2004.09297*, 2020.

[5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," *arXiv preprint arXiv:2106.09685*, 2021.

[6] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," *arXiv preprint arXiv:2305.14314*, 2023.

[7] N. Ljubešić and D. Lauc, "BERTić - The transformer language model for Bosnian, Croatian, Montenegrin and Serbian," *arXiv preprint arXiv:2104.09243*, 2021.

[8] L. Terčon and N. Ljubešić, "CLASSLA-Stanza: The next step for linguistic processing of South Slavic languages," *arXiv preprint arXiv:2308.04255*, 2023.

[9] M. Škorić and N. Janković, "New textual corpora for Serbian language modeling," *arXiv preprint arXiv:2405.09250*, 2024.

[10] N. Ljubešić and F. Klubička, "{bs,hr,sr}WaC - Web corpora of Bosnian, Croatian and Serbian," in *Proceedings of the 9th Web as Corpus Workshop (WaC-9)*, Gothenburg, Sweden, 2014, pp. 29–35.

[11] https://wandb.ai/mostafaibrahim17/ml-articles/reports/The-Answer-Key-Unlocking-the-Potential-of-Question-Answering-With-NLP--VmlldzozNTcxMDE3

[12] https://huggingface.co/sentence-transformers/paraphrase-multilingual-mpnet-base-v2

[13] Augmenting Orbital Debris Identification with Neo4j-Enabled Graph-Based Retrieval-Augmented Generation for Multimodal Large Language Models - Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/Example-of-a-typical-decoder-only-LLM-architecture_fig3_392138444

[14] https://www.alims.gov.rs/

[15] https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct

[16] https://docs.nvidia.com/nemo-framework/user-guide/24.09/sft_peft/qlora.html

[17] https://www.kaggle.com/datasets/aleksacvetanovic/squad-sr?resource=download

[18] https://huggingface.co/datasets/saillab/alpaca-serbian-cleaned

[19] https://huggingface.co/datasets/CohereLabs/aya_dataset

[20] datatab, "alpaca-cleaned-serbian-full," Hugging Face dataset. https://huggingface.co/datasets/datatab/alpaca-cleaned-serbian-full

[21] Google Research, "ROUGE tokenization implementation (rouge/tokenize.py)." https://github.com/google-research/google-research/blob/master/rouge/tokenize.py